

Lab-16.2

Course : AI Assisted Coding

Topic : Database Design and Queries: Schema Design and SQL Generation

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

Final Optimal Prompt:

Generate SQL CREATE TABLE statements for a Student Management System with tables: Students, Courses, and Enrollments. Include primary keys, foreign keys, appropriate data types, and ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Code Screenshot :

```
1 CREATE TABLE Students (  
2     student_id INT PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     email VARCHAR(100) UNIQUE NOT NULL,  
5     phone VARCHAR(15),  
6     date_of_birth DATE  
7 );  
8  
9 CREATE TABLE Courses (  
10     course_id INT PRIMARY KEY,  
11     course_name VARCHAR(100) NOT NULL,  
12     course_code VARCHAR(20) UNIQUE NOT NULL,  
13     credits INT NOT NULL  
14 );  
15  
16 CREATE TABLE Enrollments (  
17     enrollment_id INT PRIMARY KEY,  
18     student_id INT NOT NULL,  
19     course_id INT NOT NULL,  
20     enrollment_date DATE DEFAULT CURRENT_DATE,  
21     grade VARCHAR(2),  
22  
23     -- Foreign Keys  
24     FOREIGN KEY (student_id) REFERENCES Students(student_id)  
25         ON DELETE CASCADE  
26         ON UPDATE CASCADE,  
27  
28     FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
29         ON DELETE CASCADE  
30         ON UPDATE CASCADE,  
31  
32     -- Prevent duplicate enrollments  
33     UNIQUE (student_id, course_id)  
34 );
```

Output Screenshot:

```
SQL ▾ < 1 / 1 > 1 - 0 of 0 
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(15),
  date_of_birth DATE
);

SQL ▾ < 1 / 1 > 1 - 0 of 0 
CREATE TABLE Courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(100) NOT NULL,
  course_code VARCHAR(20) UNIQUE NOT NULL,
  credits INT NOT NULL
);

SQL ▾ < 1 / 1 > 1 - 0 of 0 
CREATE TABLE Enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  enrollment_date DATE DEFAULT CURRENT_DATE,
  grade VARCHAR(2),

  -- Foreign Keys
  FOREIGN KEY (student_id) REFERENCES Students(student_id)
  ON DELETE CASCADE
  ON UPDATE CASCADE,
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The database schema design task highlights the importance of structuring data efficiently using normalization principles. By separating Students, Courses, and Enrollments into different tables, redundancy is minimized and data consistency is maintained. The use of primary keys uniquely identifies each record, while foreign keys establish relationships between tables, ensuring referential integrity. Following 1NF, 2NF, and 3NF rules eliminates duplicate and dependent data. This design improves scalability and makes future updates easier, forming a strong foundation for a reliable Student Management System.

Task-02

Final Optimal Prompt:

Generate SQL INSERT statements to populate the Students, Courses, and Enrollments tables. Add at least 3 students, 4 courses, and 5 enrollment records. Also include SELECT queries to verify the inserted data.

Code Screenshot :

```
36
37 -- Students
38 INSERT INTO Students (name, email) VALUES
39 ('Sabir Mohammed', 'sabir@gmail.com'),
40 ('Rahul Sharma', 'rahul@gmail.com'),
41 ('Anita Reddy', 'anita@gmail.com');
42
43 -- Courses
44 INSERT INTO Courses (course_name, course_code, credits) VALUES
45 ('Database Management', 'DB101', 4),
46 ('Operating Systems', 'OS102', 3),
47 ('Data Structures', 'DS103', 4),
48 ('Computer Networks', 'CN104', 3);
49
50 -- Enrollments
51 INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES
52 (1, 1, '2026-01-10'),
53 (1, 2, '2026-01-12'),
54 (2, 1, '2026-01-15'),
55 (2, 3, '2026-01-18'),
56 (3, 4, '2026-01-20');
57
58 -- =====
59 -- VERIFY DATA
60 -- =====
61
62 SELECT * FROM Students;
63 SELECT * FROM Courses;
64 SELECT * FROM Enrollments;
```

Output Screenshot:

SQL ▾					< 1 / 1 > 1 - 3 of 3			
student_id	name	email	phone	date_of_birth				
1	Sabir Mohammed	sabir@gmail.com	NULL	NULL				
2	Rahul Sharma	rahul@gmail.com	NULL	NULL				
3	Anita Reddy	anita@gmail.com	NULL	NULL				

SQL ▾				< 1 / 1 > 1 - 4 of 4			
course_id	course_name	course_code	credits				
1	Database Management	DB101	4				
2	Operating Systems	OS102	3				
3	Data Structures	DS103	4				
4	Computer Networks	CN104	3				

SQL ▾					< 1 / 1 > 1 - 5 of 5			
enrollment_id	student_id	course_id	enrollment_date	grade				
1	1	1	2026-01-10	NULL				
2	1	2	2026-01-12	NULL				
3	2	1	2026-01-15	NULL				
4	2	3	2026-01-18	NULL				
5	3	4	2026-01-20	NULL				

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The insertion of sample records demonstrates how data is populated into relational tables while maintaining integrity constraints. By adding students, courses, and enrollments, relationships between tables are clearly established. The use of valid foreign keys ensures that enrollment records correspond to existing students and courses. SELECT queries help verify that data has been inserted correctly and allow users to inspect table contents. This task emphasizes the importance of accurate data entry and validation, which is essential for maintaining a consistent and functional database system.

Final Optimal Prompt:

Write SQL queries to perform CRUD operations on the Student Management System:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record

Also include SELECT queries to verify the changes.

Code Screenshot :

```
63  -- =====
64  -- UPDATE
65  -- =====
66
67  UPDATE Students
68  SET email = 'sabir_new@gmail.com'
69  WHERE student_id = 1;
70
71  SELECT * FROM Students WHERE student_id = 1;
72
73  -- =====
74  -- DELETE
75  -- =====
76
77  DELETE FROM Enrollments
78  WHERE enrollment_id = 3;
79
80  SELECT * FROM Enrollments;
81  SELECT * FROM Enrollments WHERE enrollment_id = 3;
```

Output Screenshot:

SQL ▾< 1 / 1 > 1 - 1 of 1📄↶↷⌕

student_id	name	email	phone	date_of_birth
1	Sabir Mohammed	sabir_new@gmail.com	NULL	NULL

SQL ▾< 1 / 1 > 1 - 0 of 0📄↶↷⌕

SQL ▾< 1 / 1 > 1 - 4 of 4📄↶↷⌕

enrollment_id	student_id	course_id	enrollment_date	grade
1	1	1	2026-01-10	NULL
2	1	2	2026-01-12	NULL
4	2	3	2026-01-18	NULL
5	3	4	2026-01-20	NULL

SQL ▾< 1 / 1 > 1 - 0 of 0📄↶↷⌕

SELECT * FROM Enrollments WHERE enrollment_id = 3;

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The CRUD operations task illustrates the fundamental database actions: Create, Read, Update, and Delete. Retrieving courses shows how data can be accessed efficiently, while updating a student's email demonstrates modification of existing records. Deleting an enrollment record highlights how unwanted data can be safely removed. Verification using SELECT queries ensures that changes are correctly applied. This task reinforces the importance of using conditions in UPDATE and DELETE statements to avoid unintended data loss and ensures proper control over database operations.

Task-04

Final Optimal Prompt:

Write SQL aggregate queries to:

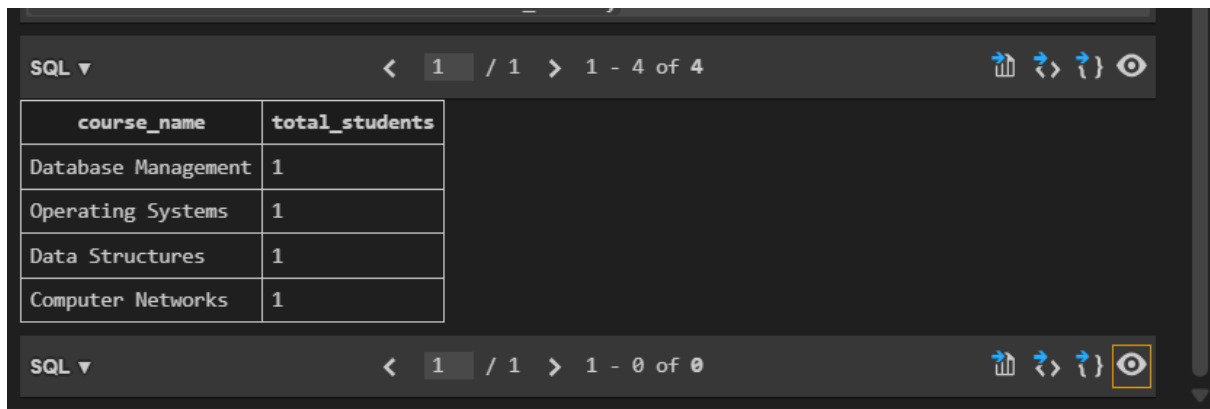
- Count the number of students enrolled in each course
- Display courses with more than one student enrolled

Ensure correct use of GROUP BY and HAVING clauses.

Code Screenshot :

```
83  SELECT
84      c.course_name,
85      COUNT(e.student_id) AS total_students
86  FROM Enrollments e
87  JOIN Courses c ON e.course_id = c.course_id
88  GROUP BY c.course_id, c.course_name;
89
90
91  SELECT
92      c.course_name,
93      COUNT(e.student_id) AS total_students
94  FROM Enrollments e
95  JOIN Courses c ON e.course_id = c.course_id
96  GROUP BY c.course_id, c.course_name
97  HAVING COUNT(e.student_id) > 1;
```

Output Screenshot:



course_name	total_students
Database Management	1
Operating Systems	1
Data Structures	1
Computer Networks	1

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The aggregate queries task focuses on summarizing and analyzing data using functions like COUNT along with GROUP BY and HAVING clauses. Counting the number of students per course provides useful insights into course popularity, while filtering courses with more than one student demonstrates the use of HAVING for grouped conditions. This task highlights the difference between WHERE and HAVING clauses and shows how aggregated data can support decision-making. It is essential for reporting and analytics in real-world database applications.

Tack-05

Final Optimal Prompt:

Analyze and optimize the following SQL query by replacing subqueries with JOINS. Provide both the original and optimized queries and ensure both return the same results:

```
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
```

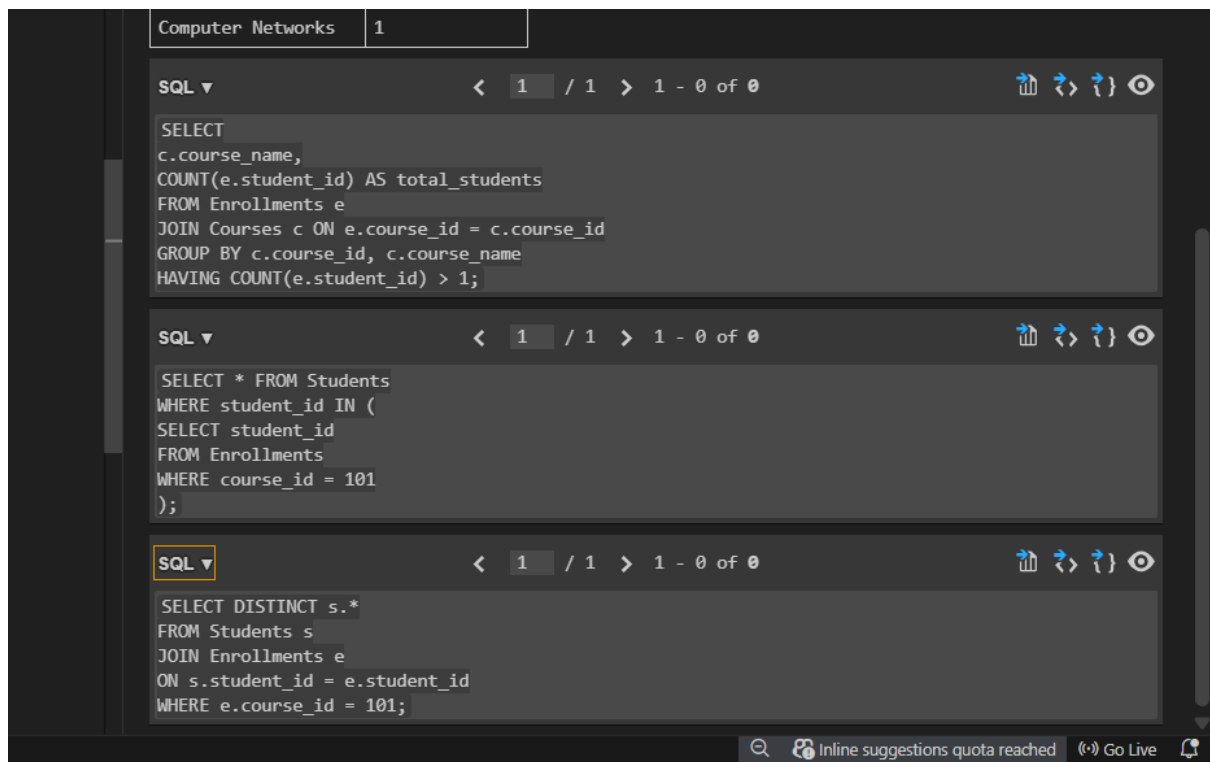
Code Screenshot :

```

99
100 SELECT * FROM Students
101 WHERE student_id IN (
102     SELECT student_id
103     FROM Enrollments
104     WHERE course_id = 101
105 );
106
107
108
109 SELECT DISTINCT s.*
110 FROM Students s
111 JOIN Enrollments e
112 ON s.student_id = e.student_id
113 WHERE e.course_id = 101;

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The query optimization task demonstrates how replacing subqueries with JOIN operations can improve performance and readability. The original query used a subquery with the IN clause, which may be less efficient for large datasets. By converting it into a JOIN-based query, data retrieval becomes faster and more direct. The use of DISTINCT ensures duplicate records are avoided. This task highlights the importance of writing efficient queries, especially when dealing with large-scale databases, and shows how optimization techniques can enhance overall system performance.
